

Introducción al Desarrollo Web con JavaScript

Laboratorio de Programación

UNPA – UACO

Ing. Jose Rasjido

AGENDA

- ✓ **Introducción**
- ✓ **Gramática y Tipos**
- ✓ **Funciones**
- ✓ **Arreglos**
- ✓ **Bibliografía**

Introducción

¿Qué es JavaScript?

- Es un lenguaje de programación de propósito general.
- Es un lenguaje interpretado y multiparadigma que soporta los estilos: funcional, orientado a objetos e imperativo.
- La programación orientada a objetos está basada en prototipos.
- Entre otras cosas, permite enriquecer los documentos HTML, pudiendo generar contenido dinámico, personalizable e interactivo.

Introducción

Especificaciones oficiales

- ECMA International se encarga de mantener la especificación oficial.
- En Junio de 1997 se publica la primera edición del estándar ECMA-262.
- En Junio de 1998, se realizan modificaciones para adaptarlo al estándar ISO/IEC-16262 y se creó la segunda edición.
- Actualmente la versión estable de JavaScript es la especificación ECMA-262 15th edición, junio 2024.
- La especificación EcmaScript-2025 se encuentra en etapa de versión borrador => <https://tc39.es/ecma262/>

Introducción

Características de JavaScript

- Es un lenguaje interpretado y de tipado dinámico.
- Orientado a Objetos basado en prototipos.
- Asíncrono y no bloqueante.
- Multiplataforma.
- Gran ecosistema. Gran variedad de herramientas librerías, frameworks y documentación.

Introducción

Características de JavaScript, para el Desarrollo Web

- Manipulación del DOM.
- Interactividad del Usuario.
- Comunicación Asíncrona.
- Compatibilidad con CSS.
- Soporte de API del Navegador.
- Desarrollo Front-end y Back-end.

Introducción

Limitaciones de JavaScript

- Seguridad: El código JavaScript se ejecuta en el cliente. Es vulnerable a ataques como Cross-Site Scripting (XSS).
- Dependencia del Navegador: La ejecución puede variar entre diferentes navegadores, aunque esto se ha mitigado con el tiempo.
- Manejo de Tipos Débil: La tipificación dinámica puede llevar a errores inesperados y dificultades en el manejo de tipos de datos.

Introducción

Limitaciones de JavaScript

- Rendimiento: Aunque ha mejorado, JavaScript puede ser más lento que lenguajes compilados en operaciones intensivas.
- Depuración Compleja: Detectar y corregir errores en JavaScript puede ser difícil, especialmente en código asíncrono o mal estructurado.
- Soporte de Módulos: Históricamente, la gestión de módulos fue limitada, aunque se ha mejorado con ECMAScript 6 (ES6) y herramientas de empaquetado.

Introducción

Vinculación con los documentos HTML

- 1) Incluir el código dentro del documento.
- 2) Vinculando archivos externos con el documento.
- 3) Embeber el código en los elementos HTML mediante el uso de eventos.

Introducción

1) Código JavaScript dentro del documento HTML

Mediante el elemento `<script type=" " >` en cualquier parte del documento.

```
<head>
```

```
    <script type="text/javascript">  
        alert('Hola mundo');
```

```
    </script>
```

```
</head>
```

Se hace uso del atributo `type` para indicar el tipo de script.

Introducción

2) Vincular *scripts* externos

Mediante el elemento `<script>` se vincula el archivo externo, haciendo uso del atributo `src`.

```
<head>
```

```
    <script type="text/javascript" src="script.js">
```

```
    </script>
```

```
</head>
```

Cada `<script>` puede enlazar un único archivo, pero se pueden incluir varios dentro del documento.

Introducción

3) Embeber JavaScript en los elementos

Se puede invocar código JavaScript, desde los eventos de los elementos HTML.

```
<input type="button" onclick="alert('click')"/>
```

Se pueden asociar eventos a los elementos HTML desde un JavaScript externo, y no tener la necesidad de incluir código en las etiquetas.

AGENDA

- ✓ Introducción
- ✓ **Sintaxis, Gramática y Tipos**
- ✓ Funciones
- ✓ Arreglos
- ✓ Bibliografía

Gramática y Tipos

Conceptos básicos

La sintaxis de un lenguaje de programación se define como el conjunto de reglas que deben seguirse para escribir el código fuente, para que un programa se considere correcto para ese lenguaje.

La sintaxis de JavaScript es muy similar a la de otros lenguajes de programación como Java y C.

Gramática y Tipos

Conceptos básicos

- Javascript distingue entre mayúsculas y minúsculas (*case-sensitive*) y utiliza el conjunto de caracteres *Unicode*.
- Las sentencias se separan con “;”. Pero no es estrictamente necesario, si la sentencia esta en su propia línea.
- No se define el tipo de las variables. Salvo que se habilite el modo estricto.

Gramática y Tipos

Comentarios

Se definen dos tipos de comentarios. Los de una línea y los de múltiples líneas.

//Comentario de una línea

/* Comentario de
mas de una línea.
*/

Gramática y Tipos

Variables y tipos de datos

- Una variable es un contenedor para un valor. Técnicamente, una variable es una referencia a memoria, ya que el valor o datos se almacena en memoria.
- Javascript se considera un lenguaje de tipado débil y dinámico.
- El último estándar ECMAScript define nueve tipos:

Gramática y Tipos

Tipo de dato	Descripción	Ejemplo
undefined	Variable sin definir	No se asigna o inicializa con un valor
boolean	Entidad lógica	true / false
number	Valor en formato binario de 64 bits de doble precisión IEEE 754	Números entre $-(2^{53} - 1)$ y $2^{53} - 1$. Además de punto flotante.
string	Cadena de texto. La longitud de una cadena es el número de elementos que contiene.	"Cadena de texto alfanumérica"
bigint	Números enteros con precisión arbitraria	9007199254740992n
symbol	Un símbolo es un valor primitivo único e inmutable y se puede utilizar como clave de una propiedad de objeto.	factura.fecha
null	Valor nulo	null
object	Objeto. Entidad en memoria.	factura = {}
function	Función o método.	function() {}

Gramática y Tipos

Declaración de variables

Una variable se puede declarar de tres formas distintas. Haciendo uso de **var**, **let** o simplemente escribiendo el identificador o nombre.

```
x = 3;
```

```
var y = 8;
```

```
let suma = x + y;
```

La diferencia entre cada variante radica en el alcance o ámbito que tendrá la variable durante la ejecución del programa.

Gramática y Tipos

Ámbito de las variables

El ámbito o alcance de una variable, es la visibilidad que tiene la misma, dentro de un programa en ejecución.

JavaScript contempla tres ámbitos: *global*, *local* y *de bloque*.

El alcance o ámbito de bloque se introduce a partir de la versión de EcmaScript 2015.

Gramática y Tipos

Variables globales

Una variable global es aquella que ha sido definida de la siguiente manera:

- Cuando es declarada en cualquier parte del documento haciendo uso de `var` o `let` y no se declara dentro de una función.
- Cuando es declarada dentro de una función sin usar la palabra reservada `var` o `let`.

Gramática y Tipos

Variables globales

```
x = 4;
```

```
var y = 10;
```

```
function sumar(){
```

```
    suma = x + y;
```

```
    return suma;
```

```
}
```

```
console.info("la suma es "+sumar());
```

Gramática y Tipos

Variables locales

Una variable local es aquella que ha sido definida dentro de una función, haciendo uso de la palabra reservada **var** o **let**:

```
var x = 4;  
let y = 2;  
function procesar(){  
    let suma = x + y;  
    var resta = x - y;  
    producto = x*y;  
}  
procesar();
```

¿Cuáles son variables
locales y cuales globales?

Gramática y Tipos

Variables locales

Una variable local es aquella que ha sido definida dentro de una función, haciendo uso de la palabra reservada **var** o **let**:

```
var x = 4;  
let y = 2;  
function procesar(){  
    let suma = x + y;  
    var resta = x - y;  
    producto = x*y;  
}  
procesar();
```

suma y resta son variables
locales

Gramática y Tipos

Constantes

Es un valor que se define una única vez y se mantiene inmutable durante la ejecución de un programa.

Funciona exactamente de la misma manera que `let`, excepto que a `const` no se le puedes asignar un nuevo valor.

```
const PI = 3.14;
```

Gramática y Tipos

Comparación de constructores

Constructor	Redeclaración	Redefinición	Alcance
var	SI	SI	GLOBAL. Salvo si esta en una función, en este caso LOCAL.
let	NO	SI	De bloque. Puede ser global, pero no como parte del objeto Window.
const	NO	NO	De bloque. Admiten redefinición de componentes internos.

AGENDA

- ✓ Introducción
- ✓ Sintaxis, Gramática y Tipos
- ✓ **Funciones**
- ✓ Arreglos
- ✓ Bibliografía

Funciones

- Una función es un conjunto de instrucciones que se agrupan para realizar una tarea concreta, pudiendo retornar un valor específico.
- Una función consisten en la palabra reservada `function` seguida por el nombre de la función.
- Al igual que en otros lenguajes una función puede recibir parámetros.

Funciones

```
function mostrarMensaje(){  
    alert("Hola desde una función");  
}
```

```
function mostrarSuma(x, y){  
    alert("La suma es: " + ( x + y ));  
}
```

Funciones

Manipulación de eventos

- Los eventos son las acciones del navegador y las que realiza el usuario mientras visita una página. Ejemplos: enviar un formulario y mover el ratón sobre una imagen.
- JavaScript trabaja con eventos utilizando comandos denominados manejadores de eventos.
- La mayoría de los elementos HTML tiene asociado los eventos que se muestran en la siguiente tabla:

Funciones

Eventos	Lo que manipula	Elemento
onAbort	El usuario abortó la carga de la página.	<body>
onBlur	El usuario dejó el objeto.	<button>, <input>, <label>, <select>, <textarea>, <body>
onChange	El usuario cambió el objeto.	<input>, <select>, <textarea>
onClick	El usuario hizo click en un objeto.	Todos los elementos.
ondblclick	El usuario hizo doble click en un objeto.	Todos los elementos.
onFocus	El usuario activo un objeto.	<button>, <input>, <label>, <select>, <textarea>, <body>
onLoad	El objeto terminó de cargarse.	<body>
onMouseOver	El cursor se desplazó sobre un objeto.	Todos los elementos.
onMouseOut	El cursor se retiró de un objeto.	Todos los elementos.
onSubmit	El usuario envió un formulario.	<form>

Funciones

Funciones predefinidas

- **alert**: Muestra un mensaje en una ventana de dialogo.
- **confirm**: Plantea una pregunta al usuario, y devuelve **true** o **false** dependiendo de la respuesta elegida.
- **prompt**: Permite solicitar un dato al usuario.
- **parseInt**: Convierte un string en un valor entero.
- **parseFloat**: Convierte un string en un valor real.
- **isNaN**: Devuelve **true** o **false**, dependiendo si el valor es un número o no.

Funciones

Funciones flecha

- La expresión de función flecha, tiene una sintaxis mas corta que una expresión de función convencional.
- Esta técnica permite guardar una función dentro de una variable o constante. El nombre de la función pasa a ser el nombre de la variable.
- Las funciones flechas, siempre deben ser anónimas.

Funciones

Funciones flecha – Ventajas a la hora de simplificar código

- Si el cuerpo de la función sólo tiene una línea, podemos omitir las llaves (`{}`)
- Además, en ese caso, automáticamente se hace un `return` de esa única línea, por lo que podemos omitir también el `return`
- En el caso de que la función no tenga parámetros, se indica : `() =>`
- En el caso de que la función tenga un solo parámetro, se puede indicar: `e =>`
- En el caso de que la función tenga 2 ó más parámetros, se indican: `(a, b) =>`
- Si queremos devolver un objeto, que coincide con la sintaxis de las llaves, se puede englobar con paréntesis: `({name: 'Manz'})`.

Funciones

Callbacks

- Las funciones *callbacks* son llamadas hacia atrás o retrollamadas.
- Consiste en pasar una función A como parámetro de otra función B. De tal forma, que A se pueda ejecutar dentro de B.
- En Javascript son muy utilizadas para trabajar las asincronías.
- Pueden dificultar la legibilidad del código, pero se utilizan en casos específicos donde no interesa guardar una función o no se reutilizará en otra parte del código.

AGENDA

- ✓ Introducción
- ✓ Sintaxis, Gramática y Tipos
- ✓ Funciones
- ✓ Arreglos
- ✓ Bibliografía

Arreglos

Inicialización de arreglos

- Existen muchas maneras de crear un *array*. Las mas usadas son: usar *array* literales y el constructor `Array()`.

```
let arreglo = [1, 2, 3, 4];
```

```
let arreglo2 = new Array(1,2,3,4);
```

- Si el constructor se utiliza sin argumentos, se crea uno vacío.

```
let arreglo3 = new Array()           // arreglo3 = [ ]
```

Arreglos

Inicialización de arreglos

- Si se utiliza el constructor con un único argumento (un número), entonces se crea un *array* con esa longitud y con todos su valores en *undefined*.

```
var arreglo = new Array(4);
```

```
//Solo aplica cuando el argumento es número
```

Esto resulta en:

```
[ undefined, undefined, undefined, undefined ]
```

Arreglos

Agregar ítems a un *array* – push

Añade uno o más elementos al final de un arreglo.

```
var arreglo = [];
```

```
arreglo.push(3);
```

```
arreglo.push(5,7,9);
```

El arreglo quedaría:

```
[3, 5, 7, 9]
```

Arreglos

Iterar un *array* – **for in**

Permite recorrer las posiciones definidas de un arreglo.

```
let valores = [3,5,1];  
  
for(let i in valores){  
    console.log(valores[i]);  
}
```

Se debe usar con precaución, ya que no tiene el mismo comportamiento que los bucles tradicionales.

Arreglos

Iterar un *array* – **for in**

```
var arreglo = []; //Crea un nuevo arreglo vacío
```

```
arreglo[4] = 4;    //JavaScript re-dimensiona a 5
```

```
for (var i = 0; i < arreglo.length; i++) //Itera de 0 a 5
```

```
    console.log(arreglo[i]);
```

```
for (var x in arreglo) // Muestra el índice 4 e ignora 0-3
```

```
    console.log(arreglo[x]);
```

Comportamiento de
for in

Arreglos

Iterar un *array* – **for of**

En ES6, es la forma recomendada de iterar sobre los valores de una *array*.

```
for(let i of arreglo)  
    console.log(i);
```

En este caso, en cada iteración se obtiene el valor, en vez del índice.

Arreglos

Remover items de un *array* – **shift**

Remueve el primer ítem de un arreglo.

```
var arreglo = [1, 2, 3, 4];
```

```
arreglo.shift();
```

El arreglo quedaría:

```
[2, 3, 4]
```

Arreglos

Remover items de un *array* – **pop**

Remueve el último ítem de un arreglo.

```
var arreglo = [1, 2, 3, 4];
```

```
arreglo.pop();
```

El arreglo quedaría:

```
[1, 2, 3]
```

Arreglos

Remove items de un *array* – **splice**

Remueve una serie de elementos. Acepta dos parámetros: el índice de inicio y la cantidad de elementos (opcional) a eliminar.

```
var valores = [1, 2, 3, 4, 5];
```

```
var removidos = valores.splice(2,2);
```

Los arreglos quedarían:

```
valores = [1, 2, 5]
```

```
removidos = [3, 4]
```

Arreglos

Remove items de un *array* – delete

Remueve un ítem de un arreglo sin cambiar su longitud.

```
var valores = [1,2,3,4,5];
```

```
delete valores[2];
```

```
console.log(valores);
```

Los arreglos quedarían:

```
valores = [1, 2, undefined, 4, 5]
```

Arreglos

Reduciendo los valores de un *array* - `reduce()`

El método `reduce()` aplica una función sobre un acumulador y cada elemento del arreglo (de izq a der), para reducirlo a un único valor.

// Ejemplo para sumar los elementos de un arreglo:

```
[3,5,1].reduce(function(a,b){ return a + b});
```

// Ejemplo para encontrar el mínimo:

```
[3,5,1].reduce(function(a,b){
```

```
return a < b ? a : b;
```

```
}, Number.POSITIVE_INFINITY);
```

Arreglos

Mapear los valores de un *array* - `map()`

Crea un nuevo arreglo, luego de aplicar una función a cada elemento del arreglo.

```
// Elevar los números de un arreglo al cuadrado:
```

```
const cuadrado = function(numero){  
    return numero * numero;  
};
```

```
let numeros = Array(3,6,8,1,9);
```

```
let numerosAlCuadrado = numeros.map(a => cuadrado(a));
```


Arreglos

Filtrar los elementos de un *array* - `filter()`

Crea un nuevo arreglo, con todos los elementos que cumplan la condición dada por una función.

// Devolver los números pares:

```
let numeros = Array(3,6,8,1,9);
```

```
const isEven = (numero) => numero % 2 === 0;
```

```
let numerosPares = numeros.filter(a => isEven(a));
```

AGENDA

- ✓ Introducción
- ✓ Sintaxis, Gramática y Tipos
- ✓ Funciones
- ✓ Arreglos
- ✓ Bibliografía

Bibliografía

- Estándar ECMAScript, Ecma-262
 - <https://www.ecma-international.org/publications/standards/Ecma-262.htm>
- Última versión borrador de ECMA
 - <https://tc39.es/ecma262/>
- Documentación de Mozilla
 - <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>
- JavaScript – The Definitive Guide
 - David Flanagan - O'REILLY
 - ISBN: 978-0-596-80552-4

EJERCICIOS

- 1) Escribir un bucle que genere siete llamadas a console.log para mostrar el siguiente triángulo:

#

##

###

####

#####

#####

#####

EJERCICIOS

- 2) Escriba un programa que use `console.log` para imprimir los números del 1 al 100, con las siguientes excepciones, imprimir:
 - a. Fizz ” cuando el número es divisible por 3, pero no por 5.
 - b. Buzz ” cuando el número es divisible por 5, pero no por 3.
 - c. “FizzBuzz” cuando es divisible por 3 y también por 5.

- 2) Escriba una función (`range`) que reciba dos argumentos, inicio y fin , y retorne un arreglo conteniendo todos los números enteros desde inicio hasta fin.

- 3) Escriba una función (`sum`) que reciba como argumento un arreglo de números enteros (generado por `range`) y retorne la suma de esos números.